- Compute Match — Product Overview
  - What is Compute Match?
  - Who uses Compute Match?
  - The core idea in five steps
  - Key business objects
  - Important timers (defaults)
  - Technology at a glance
  - Where to get help
- Compute Match — Roles, Permissions and Access Control
  - Personas and the roles behind them
  - Feature scopes
  - Organization scoping
  - Customer confidentiality
  - Account status gates
  - Why is a menu item missing?
- Compute Match — Signing Up, Signing In and First-Time Setup
  - How sign-in works
  - Where you land after sign-in
  - First-time checks
  - NCP sign-in paths
  - How partner detection works
  - Signing out
- Compute Match — Customer Guide (Find, Request and Track GPU Clusters)
  - Customer menu
  - Step 1 — Find a Cluster to Reserve
  - Step 2 — Fill the reservation form
  - Custom requests (no matching cluster)
  - Step 3 — Track requests in My Requests
  - Step 4 — Reservation detail
  - What happens after you submit
  - Feedback
- Compute Match — NCP (Provider) Guide
  - NCP menu
  - Dashboard
  - Upload Capacities (CSV)
  - Manage reservation (split view)
  - Reservation requests (table view)
  - Reservation detail
  - Deadlines you must watch
  - Emails NCPs receive
- Compute Match — Platform Admin Guide
  - Admin menu (in order)
  - Global Capacity Explorer
  - Review Future Capacities
  - Onboard NCPs

    - Custom requests
    - Reservation requests across all NCPs
    - User operations
    - Daily alert email
    - Analytics
- Compute Match — Reservation Request Lifecycle and Status Reference
    - Two levels of status
    - Reservation request statuses
    - Offering statuses (per NCP)
    - Catalog / multi-NCP flow, step by step
    - Custom request flow
    - Capacity rules that affect requests
    - Configuration defaults
    - Allocation statuses
- Compute Match — GPU Capacity Model and CSV Import Workflow
    - GPU offering data model
    - CSV capacity import — end-to-end flow
    - CSV columns
    - NCP onboarding CSV
    - Legacy bootstrap importers
- Compute Match — Email Notifications
    - Customer emails
    - NCP emails
    - Admin and operations emails
    - Account emails
    - Magic links in detail
    - Reliability
- Compute Match — System Architecture
    - Runtime components
    - Backend architecture (DDD + hexagonal)
    - Division of labour between backend and worker
    - Data stores
    - Frontend architecture
    - Cloud platform
- Compute Match — Backend API Reference (Summary)
    - Public
    - Customers APIs
    - Providers APIs (NCP and admin)
    - Access Request APIs (only when NPN integration is enabled, otherwise 404)
    - Inventory APIs (reference data)
    - Users APIs
    - Operations APIs (role `operations`)
    - Feedback APIs
    - System APIs (Temporal worker service account)
    - Internal APIs (hidden from schema)
    - Cross-cutting behaviour
- Compute Match — Temporal Workflows and Schedules
    - Summary

    - ReservationRequestWorkflow
    - OfferingLifecycleWorkflow
    - CsvImportWorkflow
    - ProviderOnboardingWorkflow
    - Salesforce workflows
    - Retry policies (`shared/workflows/workflow_common/retry.py`)
    - Viewing workflows locally
- Compute Match — Identity and Keycloak
    - Realm and clients
    - Roles
    - Custom extensions (`keycloak-extensions/`)
    - Themes
    - Organizations
    - Local Keycloak
- Compute Match — Analytics with Superset
    - Access
    - Dashboard 1 — Compute MatchMake System Metrics (marketplace)
    - Dashboard 2 — Feedback
- Compute Match — Local Development Guide
    - Prerequisites
    - First run
    - Demo accounts
    - Development modes
    - Make targets
    - Local URLs and ports
    - Configuration files
    - Everyday loop
    - Common problems
- Compute Match — Environments, Infrastructure, Delivery and CI
    - Environments
    - Google Cloud infrastructure (Terraform)
    - Helm
    - Application release train
    - GitLab CI
    - Git workflow (CONTRIBUTING.md)
- Compute Match — Security, Observability, Salesforce Integration and Testing
    - Security model
    - Observability
    - Salesforce integration (ADR-001)
    - Testing
- Compute Match — Glossary, FAQ and Troubleshooting for Users
    - Glossary
    - Frequently asked questions
    - What's New — October 2026 release (v1.3.0)

Compute Match — Product Overview

What is Compute Match?

Compute Match, also called the **Compute MatchMake System** (internal project key **NCPMS**, code name **Clustruffle**), is an NVIDIA web platform that connects **customers** who need GPU compute capacity with **NVIDIA Cloud Providers (NCPs)** who own and operate GPU infrastructure.

In one sentence: customers find GPU clusters, send reservation requests, and NCPs accept, negotiate, and contract those requests, while NVIDIA Platform Admins oversee the marketplace.

Who uses Compute Match?

There are three personas:

Persona	Who they are	What they do
Customer (also called **Offtaker** or requester)	An organization that needs GPU compute	Browse available GPU clusters, submit reservation requests (to one or several NCPs), submit custom requests, track and cancel requests
NCP (NVIDIA Cloud Provider / GPU Compute Provider)	An organization that operates GPU data centers	Upload GPU capacity via CSV, review incoming reservation requests, accept, reject, counteroffer, and confirm contracts
Platform Admin (NVIDIA operations / business admins)	NVIDIA staff running the marketplace	Explore global capacity, review future capacity uploads, onboard NCPs, approve NCP access requests, broker custom requests, extend deadlines, view analytics dashboards

A single user can hold both customer and NCP access; they switch between them with **Switch profile to** in the account menu.

The core idea in five steps

1. **NCPs publish capacity.** An NCP uploads a CSV describing how many GPUs of which platform (for example H100, B200, GB200) are available, in which country and city, and from which date.
2. **Customers search.** A customer opens **Find a Cluster to Reserve**, filters by platform, region, country, GPU count, tenure, and earliest start date, and sees matching clusters on a world map and as cards.
3. **Customers request.** The customer submits a **reservation request** for one cluster, or for up to 3 clusters from different NCPs at once (multi-NCP request). If nothing fits, they submit a **custom request** that NVIDIA admins broker manually.
4. **NCPs respond.** Each selected NCP receives an email with a secure **magic link**. The NCP accepts (moving the request to **In Process**), rejects, or counteroffers, and finally clicks **Confirm Contract**.

5. **Contract.** The first NCP to contract wins; the other NCPs' offerings on the same request are cancelled as superseded. Capacity is allocated, and the deal is synced to Salesforce as an Offtake record.

Key business objects

Object	Meaning
GPU Offering / GPU Capacity	A cluster an NCP makes available: GPU platform, hosting provider, region, country, city, GPU count and availability date windows
Capacity type	`AVAILABLE` (current capacity, starts within 90 days of upload, published right away) or `FUTURE` (starts more than 90 days out, needs Platform Admin review)
Reservation Request	A customer's request to reserve GPU capacity
Reservation Request Offering	One NCP's slice of a reservation request; a multi-NCP request has one per NCP, each with its own status and deadlines
Proposal	One version of the reservation terms (dates, GPU count, platform, region, price). The customer's original request is the first proposal; NCP counteroffers add more
Custom Request	A request without a specific cluster; NVIDIA admins move it through Open, In Review, Matched or Unserviceable
Allocation	Capacity consumed once a request is contracted
Import Job	One CSV capacity upload and its processing status
Access Request	A pending request from a partner (for example via the NPN portal) to join as an NCP

Important timers (defaults)

- An NCP has **7 days** to accept a request routed to them (acceptance window).
- After accepting, the NCP and customer have **45 days** to reach a contract (contract negotiation window).
- Reminder emails are sent **3 days** before each deadline.
- A magic link is valid for **1 hour**, and can be regenerated for up to **7 days** after routing.
- Custom requests never expire automatically.
- Platform Admins can extend either window by **1 to 200 days**.

Technology at a glance

- **Frontend:** Next.js 15, React 19, Ant Design 6, Tailwind CSS 4, NextAuth 5 with Keycloak.
- **Backend:** Python 3.13, FastAPI, Domain-Driven Design with hexagonal architecture, SQLAlchemy, PostgreSQL, Alembic migrations.
- **Workflows:** Temporal (Temporal Cloud in shared environments) with a Python workflow worker.
- **Identity:** Keycloak with NVIDIA Starfleet SSO, custom Keycloak extensions.
- **Analytics:** Apache Superset dashboards.

- **Platform:** Google Cloud — GKE Autopilot, Cloud SQL PostgreSQL, Memorystore Redis, Secret Manager; deployed with Terraform and Helm through GitLab CI.
- **Integrations:** Salesforce (Offtake sync and partner-domain sync), email notifications.

Where to get help

- In the app: **Page Help** (the `?` icon) for context help and **User Manual** (book icon) for the full role-filtered manual.
- Support email for access problems: computematchmake@nvidia.com.
- Developer docs: the `docs/` folder in the repository, also published as the Compute Match documentation site.

Compute Match — Roles, Permissions and Access Control

Personas and the roles behind them

Compute Match uses Keycloak realm roles. The frontend turns roles into feature **scopes**, and the backend checks scopes on each API call.

Persona	Keycloak roles (examples)	Notes
Customer / Offtaker	`customer`, `customer_account_manager`, Starfleet variants such as `customer_consumer`, `customer_tech_admin`	Self sign-up users get `default-roles-customer`
NCP business user	`provider`, `provider_business_user`	Can manage inventory and act on reservations. Members of the Keycloak group `offering-validator` receive magic-link emails
NCP basic user	`provider_basic_user`	Read-only NCP access; never receives magic links
Platform Admin	`platform_business_admin`, `platform_business_admin_super_user`, `platform_business_admin_ops_user`, `platform_customer_provider_am`	Cross-NCP visibility, review and onboarding
Account managers	`account_manager_consumer`, `account_manager_provider`	The only roles allowed to belong to more than one organization
Operations / system	`operations`, `systems`	User activation, service accounts
Analytics	`superset_users`, `superset_admins`, `superset_dashboard_viewers`	Superset access

Feature scopes

Scopes are defined in `backend/infrastructure/driving/http/routers/scopes.py` and `frontend/src/services/auth/accessControl/scopes.ts`.

Scope	Allows
`view_capacity`	Browse GPU offerings and availability, read reference data
`create_reservation_request`	Submit reservation and custom requests
`requester_view_reservation_request`	See your own requests (My Requests)

Scope	Allows
`requester_manage_reservation_request`	Cancel your own requests
`send_request_multiple_provider`	Use the **Send request to multiple NCP's** toggle
`view_provider_inventory`	NCP views own inventory
`manage_inventory`	Upload capacities, add or delete offerings (gates `/provider/gpu-capacities-upload`)
`approver_view_reservation_request`	NCP sees reservation requests
`approver_manage_reservation_request`	NCP accepts, rejects, contracts (gates the split **Manage reservation** view)
`approver_view_custom_reservation_request` / `approver_manage_custom_reservation_request`	View and move custom requests (Platform Admin)
`review_capacity`	Accept or reject future capacity uploads (gates `/provider/review-future-capacities`)
`manage_ncp`	Onboard NCPs and review access requests (gates `/provider/ncp-onboarding`)
`global_capacity_view`	Global Capacity Explorer (gates `/provider/global-capacity`)
`accept_contract_extension`	Extend acceptance or contract deadlines
`notification:send`, `capacity:sync`, `reservation-request:write`	Service-account scopes used by the Temporal worker

Organization scoping

- Every user belongs to a Keycloak **Organization**. NCP offerings carry `provider_organization_id`, so NCP users only see their own organization's inventory and requests.
- Platform Admins see across all NCPs.
- Only account-manager roles may be in multiple organizations (enforced by the `multi-org-policy` Keycloak extension).
- One organization exists per Salesforce partner account; NCP organization names come from the Salesforce account name.

Customer confidentiality

NCPs may see a customer's organization name blurred and labelled **Confidential** until engagement. Contracted prices are only visible to NCPs, platform account managers and admins.

Account status gates

After login, the backend middleware and frontend middleware check, in order:

1. **Access denied** — identity verified but not authorized (for example personal email domain, missing role). User is pinned to `/access-denied`.
2. **Access pending** — the user's email domain belongs to an NVIDIA partner but the NCP has not been onboarded yet. An access request is created and the user waits on `/access-pending` until a Platform Admin approves.
3. **User status PENDING** — account awaiting activation; user sees `/user-status-pending` ("Thanks for Signing up").
4. **Profile incomplete** — user must fill **Complete Your Profile** (first name, last name, organization name, phone).
5. **Terms not accepted** — a blocking modal requires scrolling the Terms and Conditions, ticking "I have read and agree to the Terms and Conditions" and clicking **Accept & Continue**.

User account statuses are `ACTIVE` and `PENDING`.

Why is a menu item missing?

Menu items appear only when you hold the matching scope. For example, **Upload Capacities** needs `manage_inventory`; **Global Capacity Explorer** needs `global_capacity_view`. If something is missing, ask your Platform Admin to grant the role. NCP basic users (`provider_basic_user`) are read-only by design and are sent to the table view instead of the split Manage reservation view.

Compute Match — Signing Up, Signing In and First-Time Setup

How sign-in works

Compute Match uses **Starfleet**, NVIDIA's single sign-on service, through **Keycloak**. Sign up and Login use the same flow.

1. Open the home page and click **Get Started**, **Sign up** or **Login**.
2. On the `/login` page enter your **Work email** and click **Login**.
3. Personal email domains (Gmail, Yahoo, Hotmail and similar) are blocked with an explanation. Use your work or enterprise email.
4. You are redirected to Starfleet and your organization's identity provider (for example NVIDIA corporate login or your company SSO).
5. After authentication you return to Compute Match. You may briefly pass through several authentication pages; this is normal.

Technically, the frontend uses NextAuth 5 with the Keycloak client `customer-web` in the `customer` realm, and passes `kc_idp_hint=starfleet` to send users to the Starfleet identity broker.

Where you land after sign-in

Persona	Landing page
Customer	**Find a Cluster to Reserve** (`/customer/reservation-request/available-clusters`)
NCP	**Dashboard** (`/provider/dashboard`)
Platform Admin	**Global Capacity Explorer** (`/provider/global-capacity`) when available, otherwise the provider dashboard

First-time checks

After the first sign-in you may see one or more of these screens:

- **Complete Your Profile** (customers, `/customer/additional-user-details`) or **Complete Your NCP Profile** (NCPs, `/provider/additional-user-details`). Fill First Name, Last Name, Organization Name and Phone Number (digits only; `+1` is pre-filled) and click **Submit**.
- **Terms and Conditions modal**. Scroll to the end, tick the checkbox and click **Accept & Continue**, or **Sign out**. Acceptance is stored as a timestamp on your user (`POST /api/v1/users/me/terms-acceptance`).
- **Thanks for Signing up** (`/user-status-pending`). Your account is pending activation; you will get an email when activated.
- **Your access is being reviewed** (`/access-pending`). Your company is an NVIDIA partner but its NCP organization is not onboarded yet. A Platform Admin must approve your access request. Once approved, the page shows **Your**

access has been approved while the session refreshes.
- **Access denied** (`/access-denied`). Your identity was verified but you are not authorized. The page shows your email and the reason. Contact computematchmake@nvidia.com or sign out and use another work account.

NCP sign-in paths

Welcome email

New NCP users created by NCP onboarding receive a `provider_welcome` email with a direct login link.

Magic link (respond to a reservation)

When a reservation request is routed to an NCP, users in that NCP organization who have the `provider_business_user` role and belong to the `offering-validator` group get an email with a magic link.

1. The link is a Keycloak action token created by the custom `accept-reservation-token` extension.
2. Keycloak validates the token and a one-time nonce with the backend (`POST /api/v1/offering/validate-nonce`), blocks the link if a different user is already logged in, creates a session and redirects.
3. The browser lands on `/automatic-login?from=magic-link&callbackUrl=...` , which skips the email page and reuses the SSO session, then opens **Manage reservation** so the NCP can **Accept**.
4. A magic link is valid for 1 hour. NCPs or admins can resend one (`POST /api/v1/provider/offering/{offeringId}/resend-magic-link`) within 7 days of routing.

NPN portal

NVIDIA Partner Network (NPN) members can reach Compute Match directly from the NPN portal (feature flag `NPN_PORTAL_INTEGRATION_ENABLED`). They still pass through the work-email page and Starfleet. If their partner organization is not onboarded yet, an access request with source `NPN_PORTAL` is created and they wait on the access-pending page.

How partner detection works

- Compute Match keeps a **partner domain** mirror of email domains belonging to NVIDIA partners, synced from Salesforce twice a day.
- Domain lookup statuses are `PARTNER` , `NOT_PARTNER` and `PENDING` .
- Personal mail providers can never be partner domains.
- When a user from a partner domain signs up and their NCP is not onboarded, `POST /api/v1/user` creates an **access request** (statuses `PENDING` , `APPROVED` , `REJECTED` ; sources `NPN_PORTAL` , `SIGNUP`).
- A Platform Admin approves or rejects it under **Onboard NCPs → Review Onboarding Requests**. Approval provisions the organization and user in Keycloak; rejection emails the reason. A rejected applicant can apply again, which creates a new request.

Signing out

Use **Logout** in the account menu. If your session cannot be refreshed, the app shows a recovery screen and performs a full Keycloak logout.

Compute Match — Customer Guide (Find, Request and Track GPU Clusters)

This guide is for customers (also called Offtakers): organizations that want to reserve GPU capacity from NVIDIA Cloud Providers (NCPs).

Customer menu

Click your name in the top-right to open the account menu:

Menu item	Route
Request a Cluster	`/customer` (redirects to Find a Cluster to Reserve)
My requests	`/customer/reservations`
Logout	—

The header also has **Page Help** (`?`), **User Manual** (book icon), **Share Feedback**, and the **What's New** bell.

Step 1 — Find a Cluster to Reserve

Route: `/customer/reservation-request/available-clusters`. This is the customer landing page. It shows GPU clusters from all NCPs on an interactive world map plus capacity cards.

Filters

Filter	What it does
Search	Free text across platform, region, country, GPU model
Platform	GPU model (for example H100, B200). Select this first: Tenure and Earliest Start Date unlock only after a platform is chosen
Geographical Region	Broad region such as North America, Europe, Asia Pacific
Country	Cascades from Geographical Region
No. of GPUs	Minimum available GPUs
Tenure	Reservation length band: `0–1` , `1–2` , `2–3` , `3–4` , `4–5` , `5–6` , `6+` years. Requires Platform
Earliest Start Date	Must be after today

Each card shows City, GPU count, and **Earliest Available Date**. Cards are labelled **Current** (available capacity) or **Future** (capacity starting later). An amber note means the cluster already has pending requests. If the platform admins set a pending-request cap, oversubscribed clusters are hidden.

Selecting clusters

1. Narrow the list with filters.
2. Click a map bubble or a card.
3. Optional: turn on **Send request to multiple NCP's** (needs the `send_request_multiple_provider` scope). Checkboxes appear; you can select up to **3** clusters (configurable). Selecting more shows a warning.
4. Click **Request a Cluster** to open the reservation form with your selection pre-filled.

Changing the Platform while cards are selected asks you to confirm clearing the selection.

Step 2 — Fill the reservation form

The form opens in a modal with three sections.

Cluster Details

- Pre-filled and read-only when you picked a cluster.
- With several clusters, they are shown as Option 1, Option 2… separated by "or", with an amber banner: these are **alternatives**, and your request will be matched with **one** cluster based on NCP availability.
- For custom requests, GPU Model and Geographical Region are required and Country is optional.

Customer Details

Field	Required	Rules
Organization Name	Yes	Dropdown with suggestions, or "Other" with a new unique name
Organization URL	No	Valid URL
Contact Email	No	Valid email, max 250 characters
Purpose	No	Max 1,000 characters
Terms & Conditions	Yes	Pre-accepted on submit

GPU Reservation Period

Field	Required	Rules
GPUs	Yes	Whole number of at least 1
Preferred start date / Earliest Start Date	Yes	After today
Duration (months)	When not using tenure	At least 1 month
Tenure	When you came in with a tenure filter	One of the tenure bands

If you ask for more GPUs or an earlier date than the cluster shows, a yellow warning appears; you can still submit and negotiate with the NCP.

Form buttons

- **Submit reservation request** validates and submits (`POST /api/v1/reservation-request`). On success you see “Successfully created Reservation Request” and land on the request detail page.
- **Clear Form** resets fields but keeps the cluster pre-fill.
- Clicking outside or pressing Esc after typing asks why you are leaving, or asks you to **Keep editing** or **Discard**. The × and **Cancel** buttons close immediately.

Custom requests (no matching cluster)

If nothing fits, click **Submit a custom request** (banner under the filters) or **Submit Custom Request** (empty state). Your current filters are carried over as hints. A custom request is not sent to a specific NCP; NVIDIA Platform Admins broker it manually and move it through:

Open → **In Review** → **Matched** or **Unserviceable**. You can cancel it while it is Open or In Review. Custom requests never expire. You get an email at each step.

Step 3 — Track requests in My Requests

Route: `/customer/reservations` .

- Columns: Request ID (shown as “REQ-1 - Status”), Customer, Contact Email, Platform, Geographical Region, Country, Requested GPUs, Total GPUs, Requested Date, Start/End Date, Duration (Hours), Status.
- Multi-NCP requests show as a grouped row with one child row per NCP.
- The date under the status chip is when it moved to In Process, Contracted, Rejected, Expired or Cancelled. Rejected rows have an info icon with the NCP’s remarks.
- Filters: Status, Hosting Provider (hidden for multi-provider), Platform, Geographical Region, Country, Sort by request date, **Clear All**.
- With no requests you see **No requests found** with a button to create one.

Step 4 — Reservation detail

Route: `/customer/reservations/{reservationId}` .

- A headline states the status, for example “Your Reservation request is awaiting NCP acceptance” or “Contract Complete!”, with the expiry date when relevant.
- **Reservation Summary**: organization, contact email, period, GPU count, platform, region, country, rejection remarks, terms acceptance.
- **Proposal History**: every proposal (yours and NCP counteroffers), grouped by region when several NCPs are involved.
- **Cancel**: available while Awaiting NCP Acceptance, Awaiting Provider/Customer Approval, In Process or Expired. Confirm in the modal; you may give a reason. All involved NCPs are notified by email.
- You cannot accept or reject NCP proposals on this page; the NCP confirms the contract.

What happens after you submit

1. You get a “request received” email.
2. Each selected NCP gets a magic-link email and has 7 days to accept.

3. When an NCP accepts, the request goes **In Process** and you get an introduction email so you can negotiate directly. The contract window is 45 days.
4. When an NCP clicks **Confirm Contract**, your request becomes **Contracted**; other NCPs on the same request are cancelled as superseded.
5. If every NCP rejects, the request is **Rejected**. If all deadlines pass, it is **Expired** and you get one consolidated expiry email.

Feedback

After creating a request you may see “How was the Request a Cluster experience?” with 1-5 stars and **Submit Feedback**. When cancelling or leaving the marketplace you may be asked why. You can also click **Share Feedback** in the header at any time.

Compute Match — NCP (Provider) Guide

This guide is for NVIDIA Cloud Providers (NCPs): organizations that publish GPU capacity and respond to customer reservation requests.

NCP menu

Menu item	Route	Needs scope
Dashboard	`/provider/dashboard`	—
Upload Capacities	`/provider/gpu-capacities-upload`	`manage_inventory`
Manage reservation	`/provider?view=split`	`approver_manage_reservation_request`
Reservation requests	`/provider?view=table`	`approver_view_reservation_request`
Custom requests	`/provider/custom-requests`	custom request scopes (usually admins)

There is no sidebar and no in-page back button on the main NCP pages; navigate with the account menu.

Dashboard

Route: `/provider/dashboard`. A read-only overview that loads fresh on each visit.

- **Welcome header** with your name and how many requests await your action.
- **Awaiting Your Action** — requests waiting for accept, reject, counter or contract.
- **Available Capacities** — your live inventory plus your latest upload status.
- Quick links: **View all requests** (table view), **Manage Reservations** (split view), **View all uploads**, **Upload Capacity**.
- Before your first upload, both panels show a guided empty state.

Upload Capacities (CSV)

Route: `/provider/gpu-capacities-upload`.

1. Click **Download Template**.
2. Fill one row per capacity block.
3. Drag and drop or click to upload the CSV.
4. You see **Upload Successful**; processing continues in the background and appears in **Upload History**.

CSV rules

- UTF-8 CSV, at most **10 MB** and **10,000 data rows**, each cell at most 500 characters.
- Required columns: `GPU Platform Type`, `Number of GPUs Available`, `Live Token Date`, `Country`, `City`.
- Optional columns: `Region`, `end_datetime`.
- Metadata columns that are accepted but ignored: `NCP Name`, `RA/RD`, `Exemplar Cluster`, `aaS Capability`, `State`.

- The hosting provider name comes from your NCP organization, not from the CSV.
- Vendor prefixes like “NVIDIA” in the platform cell are tolerated; country names such as “USA” or “Holland” are normalized.
- Cells beginning with `=`, `+`, `-` or `@` are neutralized to prevent spreadsheet formula injection.
- Invalid rows are skipped with errors listed; the job fails only if no row succeeds.

Current versus future capacity

- Rows whose Live Token Date is **within 90 days** of upload are **current (AVAILABLE)** capacity and go live immediately after validation.
- Rows starting **more than 90 days** out are **future (FUTURE)** capacity. The job waits in **Pending Review** until a Platform Admin accepts or rejects it. You are emailed the result and any rejection reason appears in the Details column.

Upload History statuses

Processing, Pending Review, Completed, Rejected, Validation Failed. Click “See all details” for the full error list.

After uploading you may be asked “How was the Upload Capacity experience?”.

Manage reservation (split view)

Route: `/provider?view=split`. Your main workspace.

- **Left panel**: request cards with status tabs (Awaiting NCP Acceptance, Awaiting Provider Approval, In Process, Contracted, Rejected, Cancelled, Expired) and filters for Platform, Geographical Region and Country, plus search.
- **Right panel**: a capacity availability chart once Platform, Region and Country are all selected, so you can see whether you have room before acting.
- Drag cards to reorder priority (keyboard: Space or Enter to grab, arrows to move).
- Expand **View details** to see dates: **In Process Since**, **Contracted On**, **Rejected On**, **Expired On**, **Cancelled On**.

Actions on a card

Action	When	Effect
Accept	Awaiting NCP Acceptance	Offering moves to **In Process**; a 45-day contract window starts; customer receives an introduction email
Reject	Awaiting acceptance or In Process	Offering moves to **Rejected**; remarks are required and shown to the customer
Counteroffer	Before contract	Opens **Modify Details**: original request on the left, editable Proposed Start Date, Proposed End Date, Number of GPUs, Platform and Geographical Region on the right. **Reset**, **Cancel** or **Confirm**

Action	When	Effect
Confirm Contract	In Process	Offering becomes **Contracted**, capacity is allocated, the whole request is Contracted and other NCPs on the request are cancelled as superseded

Accepting or contracting can fail with “capacity not available” if your remaining capacity no longer covers the request.

Reservation requests (table view)

Route: `/provider?view=table`. A sortable table with GPU Model, Country, Geographical Region, Status and sort filters. To act, choose Platform, Region and Country and click **Manage Reservation** to open the split view with those filters. Basic NCP users only get this view.

Reservation detail

Route: `/provider/reservations/{reservationId}`. Read-only **Reservation Request History** with Requested By, dates, region, platform, GPUs, capacity, total GPU hours, and price and cost when enabled.

Deadlines you must watch

- **7 days** to accept after a request is routed to you. Reminder 3 days before. If you miss it the offering becomes **Expired**.
- **45 days** from acceptance to confirm the contract. Reminder 3 days before.
- Platform Admins can extend either window.
- Magic links expire after 1 hour; ask for a resend within 7 days of routing, or just log in and open Manage reservation.

Emails NCPs receive

Magic link (new request), resend, introduction (after accept), contracted, superseded (another NCP won), expiry reminders, expired, request cancelled by customer, future capacity approved or rejected, welcome email for new users.

Compute Match — Platform Admin Guide

Platform Admins are NVIDIA staff (roles such as `platform_business_admin`, `platform_business_admin_super_user`, `platform_business_admin_ops_user`) who run the marketplace. They use the same `/provider/*` pages as NCPs, with extra pages and cross-NCP visibility. The header shows the persona label **Platform Admin**.

Admin menu (in order)

Menu item	Route	Scope
Global Capacity Explorer	`/provider/global-capacity`	`global_capacity_view`
Upload Capacities	`/provider/gpu-capacities-upload`	`manage_inventory`
Reservation requests	`/provider?view=table`	approver view
Onboard NCPs	`/provider/ncp-onboarding`	`manage_ncp`
Review Future Capacities	`/provider/review-future-capacities`	`review_capacity`
Custom requests	`/provider/custom-requests`	custom request scopes
Dashboard (external)	Superset analytics, opens in a new tab	Superset role

Global Capacity Explorer

A read-only world view of all GPU capacity across NCPs.

- Filters: Search, **Platform** (required for the chart), Geographical Region, **NCP**, Country / Data Centre Location, Start Date, End Date.
- **Map | List** toggle. The map shows capacity hotspots; “No datacenter locations match the current filters.” when empty.
- **Available Capacity** chart over time, colour-coded per NCP (same colour for current and future). Tooltips show the month’s peak-day total, current and future split, and per-city breakdown.
- **Export to Excel** and **Export to CSV**.
- API: `GET /api/v1/provider/global-capacity-map`.

Review Future Capacities

NCP uploads that include rows starting more than 90 days out wait here.

- Tabs: **All**, **Pending**, **Accepted**, **Rejected**. Filter by **NCP**.
- Columns: NCP, File Name (click for details), Uploaded Date, Status, Actions.
- **Accept**: confirm in **Approve Future Capacity Submission**; status becomes Accepted and capacity goes live for customers.
- **Reject**: enter a reason (required, up to 500 characters) and click **Submit Rejection**; the NCP is emailed the reason.
- Nothing is accepted or rejected automatically; submissions wait until reviewed.

Onboard NCPs

1. Download the NCP onboarding CSV template.
2. Fill in the NCP organizations and users.
3. Upload. A background Temporal `ProviderOnboardingWorkflow` validates the file, creates Keycloak organizations, users, roles and the `offering-validator` group membership, and sends welcome emails.
4. Watch status in the history table. Onboarding job statuses: QUEUED, VALIDATING, VALIDATION_FAILED, ONBOARDING, COMPLETED, FAILED.

Review Onboarding Requests (access requests)

When the NPN portal integration is on, partner users whose NCP is not onboarded appear here. **Approve** provisions them like a CSV onboarding; **Reject** requires a reason and emails it. API: `/api/v1/provider/access-requests` (+ `/approve`, `/reject`).

Custom requests

Route: `/provider/custom-requests`. Table only (no split view, no NCP filter). Filter by Platform, Country, Geographical Region, Status, Sort.

Lifecycle you control: - **Open** → **In Review** (you are working on it) - **In Review** or **Open** → **Matched** (**Mark as Matched** / **Confirm Match**, optionally record the NCP, NCP region and a comment) - → **Unserviceable** (**Close as Unserviceable**) - Customers alone can **Cancel** while Open or In Review. - Matched, Unserviceable and Cancelled are final; a request never returns to Open. The customer is emailed on each change.

Reservation requests across all NCPs

The table view shows all requests platform-wide. Use the row action menu (⋮) to extend deadlines:

Action	Applies to	Effect
Extend Acceptance Duration	Awaiting NCP Acceptance (or recently expired)	Adds days to the NCP's acceptance window
Extend Contract Negotiation Duration	In Process	Adds days to the contract window

Enter 1 to 200 extra days, check the previewed new expiry date, and click **Confirm**. Extending can revive an Expired offering back to its prior active status.

User operations

Operations users (`operations` role) can activate pending users, list users, resend activation magic links and delete users through the Operations APIs (`/api/v1/...` under the Operations tag).

Daily alert email

Every day at 09:00 UTC a Temporal schedule (`in-process-requests-alert-daily`) emails Platform Admins a digest of In Process offerings approaching their contract deadline.

Analytics

The external **Dashboard** link opens Superset. See the analytics document for the dashboards (Marketplace metrics and Feedback).

Compute Match — Reservation Request Lifecycle and Status Reference

This document explains, end to end, how a reservation request moves through statuses, who triggers each change, and which timers apply.

Two levels of status

1. **Reservation request status** — the status of the whole customer request.
2. **Offering status** — the status of each NCP's slice (`ReservationRequestOffering`). A request sent to 3 NCPs has 3 offerings.

Do not confuse these with **GPU offering status** (inventory listing status: `ACTIVE` , `PENDING_REVIEW` , `REJECTED`).

Reservation request statuses

Backend enum `ReservationRequestStatus` (`backend/core/reservation_request/reservation_request_status.py`):

Backend value	UI label	Meaning
`Open`	Open	Submitted; not yet routed (custom requests start and may stay here)
`AwaitingProviderAcceptance`	Awaiting NCP Acceptance	Routed to one or more NCPs, waiting for a response
`InProcess`	In Process	At least one NCP accepted; negotiation under way
`Contracted`	Contracted	An NCP confirmed the contract (legacy `Approved` also shows as Contracted)
`Rejected`	Rejected	All NCPs rejected
`Expired`	Expired	Deadlines passed without action
`Cancelled`	Cancelled	Cancelled by the customer
`InReview`	In Review	Custom request being worked on by admins
`Matched`	Matched	Custom request matched to an NCP by admins
`Unserviceable`	Unserviceable	Custom request closed as not serviceable

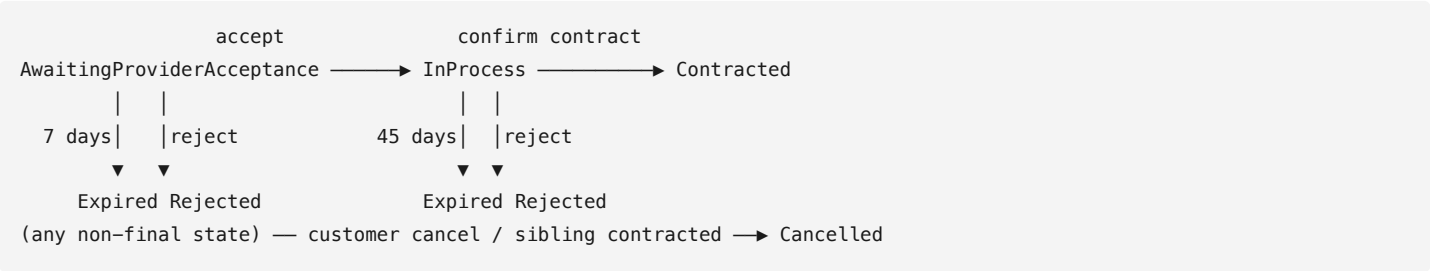
The UI also shows **Awaiting Provider Approval** and **Awaiting Customer Approval** for older proposal/counteroffer flows.

Offering statuses (per NCP)

`OfferingStatus`: `AwaitingProviderAcceptance`, `InProcess`, `Contracted`, `Expired`, `Rejected`, `Cancelled`.

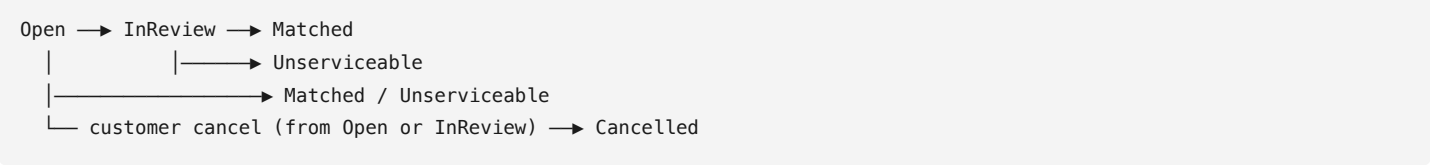
Catalog / multi-NCP flow, step by step

1. **Customer submits** (`POST /api/v1/reservation-request`). The backend creates the request and the customer's first proposal, then routes it: one offering per selected cluster (backend limit 10, UI limit 3 by default).
2. Each offering starts as **AwaitingProviderAcceptance** with `acceptance_expiry_date = now + 7 days`. The request becomes **AwaitingProviderAcceptance**.
3. Temporal starts a parent `ReservationRequestWorkflow` and one child `OfferingLifecycleWorkflow` per offering. The customer gets a "request received" email; each NCP gets a magic-link email.
4. **NCP accepts** → offering **InProcess**, `contract_expiry_date = now + 45 days`, NCP and customer receive an introduction email. The first acceptance moves the request to **InProcess**.
5. **NCP counteroffers** → a new proposal is recorded (price-only changes update in place; material changes create a new proposal in history).
6. **NCP confirms contract** → offering **Contracted**, an **Allocation** is created (status `scheduled`), the request becomes **Contracted**, and sibling offerings are **Cancelled** with "superseded" emails.
7. **NCP rejects** (remarks required) → offering **Rejected**. If all offerings are rejected, the request is **Rejected**.
8. **Timers**: if an NCP does not accept in 7 days → offering **Expired**. If no contract in 45 days → offering **Expired**. When every offering has expired, the customer gets one consolidated expiry email and the request becomes **Expired**. Reminders go out 3 days before each deadline.
9. **Customer cancels** → request **Cancelled**; NCPs are notified.
10. **Extensions**: a Platform Admin (scope `accept_contract_extension`) can extend acceptance or contract deadlines by 1–200 days; this can revive an Expired offering.



Custom request flow

A custom request has no selected cluster (`gpu_offering_id` is empty and there are no offerings).



- Only admins with `approver_manage_custom_reservation_request` move it forward.
- Only the customer cancels.
- Never returns to Open; Matched, Unserviceable and Cancelled are final.
- Custom requests never expire.

- Each transition records a timestamp and optional NCP name, NCP region and comment, and emails the customer.

Capacity rules that affect requests

- Capacity windows of the same type (AVAILABLE or FUTURE) cannot partially overlap; at most one open-ended window per type.
- Availability checks use the **upper bound** of the tenure band (for example `2–3` years is checked as 3 years) to be conservative.
- Allocations only subtract from the matching capacity type.
- An optional cap (`MAX_PENDING_REQUESTS_PER_REGIONAL_CAPACITY` , default 0 = unlimited) hides clusters with too many pending requests and rejects new requests against them.
- GPU count per request must be between 1 and 100,000.
- Contracted price must be zero or positive and is only visible to NCPs and NVIDIA admins.

Configuration defaults

From `backend/multi_provider_config.py` (environment prefix `MULTI_PROVIDER_`):

Setting	Default	Meaning
`ACCEPTANCE_EXPIRY_DAYS`	7	NCP acceptance window
`IN_PROCESS_EXPIRY_DAYS`	45	Contract negotiation window
`MAX_GPU_OFFERINGS_PER_REQUEST`	10	Backend cap on offerings per request (UI default is 3, `NEXT_PUBLIC_MAX_MULTI_NCP_SELECTION_COUNT`)
`MAX_PENDING_REQUESTS_PER_REGIONAL_CAPACITY`	0	0 means unlimited
`MAGIC_LINK_EXPIRY_SECONDS`	3600	Magic link valid for 1 hour
`MAGIC_LINK_REGENERATION_WINDOW_DAYS`	7	Resend allowed for 7 days after routing

Reminder lead time is `EXPIRY_REMINDER_LEAD_DAYS = 3` in the Temporal workflow.

Allocation statuses

`scheduled` , `provisioning` , `running` , `stopping` , `complete` .

Compute Match — GPU Capacity Model and CSV Import Workflow

GPU offering data model

A **GPU offering** (`backend/core/gpu_offering/gpu_offering.py`) represents a GPU cluster type an NCP offers:

- GPU model / platform (for example H100, H200, B200, GB200)
- Provider organization and **hosting provider** (the NCP's data-center brand)
- Description, product link (must be HTTP(S)), VRAM (positive if set)
- Interconnect: `InfiniBand` or `Spectrum-X`
- Status: `ACTIVE` (listed to customers), `PENDING_REVIEW` (awaiting admin review), `REJECTED`
- One or more **regional capacities**

A **regional capacity** ties the offering to a location: geographical region, country, address and latitude/longitude, with a list of **capacity windows**.

A **capacity window** has a date range (finite or open-ended), GPU count, delivery id, block location, a **capacity type** (`AVAILABLE` or `FUTURE`) and an AI-compute flag.

Reference data: - **Region** — a data-center location belonging to a hosting provider and a geographical region. - **Geographical region** — a broad area with a code, display name and list of ISO country codes. - **GPU model** — catalog of platform names.

CSV capacity import — end-to-end flow

1. **Upload.** An NCP with `manage_inventory` uploads a CSV on **Upload Capacities** (`POST /api/v1/provider/gpu-offerings/import` , returns 202 Accepted). The backend checks size (10 MB) and row count (10,000).
2. **Job created.** An `ImportJob` is stored with status `QUEUED` , and Temporal workflow `CsvImportWorkflow` starts with id `csv-import-{job_id}` .
3. **Validate.** The worker validates columns and rows (`VALIDATING`). If nothing is valid → `VALIDATION_FAILED` .
4. **Import.** Valid rows are imported atomically (`IMPORTING`). Failures here → `IMPORT_FAILED` .
5. **Classify.** Rows starting within 90 days of upload become `AVAILABLE` and are published immediately. Rows starting later become `FUTURE` .
6. **Review (future only).** If there are new future rows, the job moves to `PENDING_REVIEW` , admins are emailed, and the workflow waits for an `approve_import` or `reject_import` signal.
7. **Decision.** A Platform Admin with `review_capacity` accepts (`APPROVED` then `COMPLETED` , offerings activated) or rejects (`REJECTED` / `FUTURE_CAPACITY_REJECTED` , with reason).
8. **Notify.** The NCP is emailed the result.

Import job statuses

`QUEUED` , `VALIDATING` , `VALIDATION_FAILED` , `IMPORTING` , `IMPORT_FAILED` , `PENDING_REVIEW` , `APPROVED` , `REJECTED` , `FUTURE_CAPACITY_REJECTED` , `COMPLETED` .

UI labels: Processing, Pending Review, Completed, Rejected, Validation Failed.

Retry behaviour

- Import activities retry up to 3 times; validation and CSV-format errors are not retried.
- The workflow itself runs at most once, so users never get duplicate emails from replays.

CSV columns

Column	Required	Notes
GPU Platform Type	Yes	Vendor prefix such as “NVIDIA” tolerated
Number of GPUs Available	Yes	Positive integer
Live Token Date	Yes	Start of availability; decides AVAILABLE vs FUTURE
Country	Yes	Names normalized (“USA” → United States, “Holland” → Netherlands)
City	Yes	Matched against a city index with alternate names
Region	No	
end_datetime	No	End of availability; open-ended if blank
NCP Name, RA/RD, Exemplar Cluster, aaS Capability, State	No	Metadata, ignored on import

Limits: UTF-8, 10 MB, 10,000 rows, 500 characters per cell. Formula-like cells (`=` , `+` , `-` , `@`) are neutralized.

Note: the in-app User Manual still says 1,000 rows; the backend enforces 10,000 (`MAX_CSV_ROWS` in `backend/core/gpu_offering/gpu_offering_csv_import_fields.py`).

NCP onboarding CSV

Admins with `manage_ncp` upload an onboarding CSV (`POST /api/v1/provider/onboard-providers`). `ProviderOnboardingWorkflow` validates it, creates Keycloak organizations, users, roles and group memberships (30-minute timeout), then emails the initiator. Statuses: `QUEUED` , `VALIDATING` , `VALIDATION_FAILED` , `ONBOARDING` , `COMPLETED` , `FAILED` .

Legacy bootstrap importers

For seeding environments there are offline scripts in `backend/` : `import_excel_gpu_data.py` (see `EXCEL_IMPORT_README.md`), `csv_data_importer.py` , `csv_gpu_importer.py` , `generate_seed_data.py` . They use an older column layout and are separate from the production Temporal import path.

Compute Match — Email Notifications

Emails are rendered by the backend (`backend/infrastructure/driven/email/` , templates in `templates/*.html` , subjects in `subjects.json`). Most are triggered by Temporal workflow activities calling the backend's System notification APIs (`/api/v1/notification/*` , scope `notification:send`).

Customer emails

Email	When
Reservation request received	Right after you submit
Custom request received / in review / matched / unserviceable	Each custom-request step
Offering introduction	An NCP accepted; introduces you to the NCP
NCP proposed changes	An NCP counteroffered
Offering rejected	An NCP rejected your request (with remarks)
Reservation approved / contracted	An NCP confirmed the contract
Acceptance / contract expired (customer variant)	All offerings expired — one consolidated email
Reservation rejected / cancelled	Final outcomes

NCP emails

Email	When
Offering magic link (and grouped variant)	A request was routed to your organization
Magic link resend	Someone requested a new link
Offering introduction	You accepted
Expiry reminder	3 days before acceptance deadline
Contract expiry reminder	3 days before contract deadline
Offering expired / acceptance expired	Deadline passed
Offering contracted	You won the contract
Offering superseded	Another NCP contracted the same request
Reservation cancelled by customer	Customer cancelled
Future capacity approved / rejected	Admin decision on your upload
Provider welcome	You were onboarded

Admin and operations emails

Email	When
Future capacity pending review	An upload has future capacity to review
Custom request received	A customer submitted a custom request
In-process requests alert	Daily 09:00 UTC digest of In Process offerings nearing deadline
Access request submitted / notify admin	A partner is waiting for NCP access
Salesforce sync failure summary	Offtake sync failures (when sync is enabled)

Account emails

User activated, user pending activation, access request approved or rejected.

Magic links in detail

- Recipients: users of the NCP organization with role `provider_business_user` **and** in group `offering-validator`. Basic users never receive them.
- The backend asks the Keycloak extension endpoint `POST /realms/{realm}/clustruffle/create-action-token` to mint an action token containing the offering id and a nonce.
- When clicked, Keycloak validates the nonce with the backend, logs the user in and redirects with `from=magic-link`.
- Valid 1 hour; resend possible for 7 days after routing.
- Magic-link emails use the critical retry policy (10 attempts over about 25 minutes) because they are the only way an NCP learns about a request.

Reliability

- Ordinary notifications retry 5 times over about 2.5 minutes.
- Permanent failures are absorbed (and counted in metric `clustruffle_workflow_activity_abandoned`) so a workflow replay never re-sends email to users.

Compute Match — System Architecture

Runtime components

```
Customer / NCP / Admin browser
          |
   Regional Gateway (GCP load balancer + Cloud Armor)
     ├── Frontend (Next.js)  ──► Backend API, Keycloak
     ├── Backend API (FastAPI) ──► Cloud SQL PostgreSQL, Memorystore Redis, Temporal Cloud, Keycloak, Salesforce, email
     ├── Keycloak (identity, Starfleet SSO broker) ──► PostgreSQL
     └── Superset (analytics) ──► PostgreSQL (analytics schema)
Workflow worker (Temporal) ──► Backend System APIs, PostgreSQL, Temporal
```

Component	Source folder	Responsibility
Backend	`backend/`	REST API, domain logic, migrations, email, persistence, integrations
Frontend	`frontend/`	Browser application and authentication client
Shared workflows	`shared/workflows/`	Temporal workflow and activity definitions
Workflow worker	`workflow_worker/`	Runs Temporal workflows, activities and schedules
Keycloak	`keycloak/`, `keycloak-extensions/`	Identity provider, themes, realm config, custom Java extensions
Superset	`superset/`	Dashboards and reports
E2E tests	`e2e_playwright/`	Playwright + Cucumber browser tests
Local setup	`local-setup/`	Docker Compose development stack
Infrastructure	`infrastructure/`	Terraform, Helm charts, environment values
Tools	`tools/`	Maintenance and developer helpers
Python SDK	`backend-sdk-python/`	Generated OpenAPI client used by the worker

Backend architecture (DDD + hexagonal)

- `backend/core/` — the domain. Bounded contexts: `gpu_offering`, `reservation_request`, `hosting_provider`, `geographical_region`, `gpu_model`, `organization`, `user`, `allocation`, `access_request`, `provider_onboarding`, `feedback`, `audit`, `authorization`, `partner_domain`, `salesforce`. Ports (interfaces) and domain services live here; `kernels/` holds shared ids, enums and errors.

- `backend/infrastructure/driving/http/` — the driving adapter: FastAPI app factory (`main.py`), routers, auth middleware, request schemas, rate limiting.
- `backend/infrastructure/driven/` — driven adapters: SQLAlchemy database models and repositories with Alembic migrations, email, Keycloak admin, Temporal clients, Salesforce, feedback.
- The domain never imports infrastructure; adapters implement the core ports.

Division of labour between backend and worker

- The **backend owns all domain changes** (status transitions, imports, emails rendering).
- The **worker owns time**: timers, schedules, retries, fan-out. Its activities call back into backend System APIs using a Keycloak service account (`workflow-worker` client) with scopes `notification:send` , `capacity:sync` , `reservation-request:write` .

Data stores

- **PostgreSQL** (Cloud SQL in GCP) — application data, Keycloak data, analytics views (`analytics` schema for Superset), audit log (purged after about 13 months).
- **Redis** (Memorystore) — rate limiting buckets (default, mutate, auth, import, availability, feedback), optional cache, role-sync budgets.
- **Temporal** — workflow state (Temporal Cloud in shared environments, local Temporal in Docker for development).

Frontend architecture

- Next.js 15 App Router (`frontend/src/app/**/page.tsx`), React 19, Ant Design 6, Tailwind CSS 4, Roboto font.
- Authentication with NextAuth 5 + Keycloak; middleware (`frontend/src/middleware.ts` , `frontend/src/services/auth/config.ts`) protects `/customer` and `/provider` routes and enforces status gates and scopes.
- State: Zustand stores (marketplace, inventory, What's New) and SWR for server data; Axios for backend calls (`frontend/src/services/backend/*.ts`).
- Maps and charts with D3 and topojson; react-markdown for the in-app User Manual (`frontend/src/content/USER_MANUAL.md`).
- What's New content lives in `frontend/src/content/whats-new.json` .
- UI rules: `frontend/UI_STANDARDS.md` (page headers, filters, chips, Title Case buttons, WCAG 2.2 AA).

Cloud platform

Private **GKE Autopilot** clusters on Google Cloud, Cloud SQL PostgreSQL, Memorystore Redis, Secret Manager, regional Gateway load balancing, Cloud Armor, network policies. Temporal Cloud for orchestration. Terraform owns cloud resources; Helm owns workloads; GitLab CI builds and promotes immutable images.

Compute Match — Backend API Reference (Summary)

Interactive documentation runs at `http://localhost:8082/docs` locally. A snapshot is in `backend/openapi.json` (it omits Access Request and internal routes). Routers are in `backend/infrastructure/driving/http/routers/` and are mounted in `main.py`. All business APIs are under `/api/v1` (one v2 endpoint).

Public

Method	Path	Purpose
GET	`/health`	Liveness check, no auth

Customers APIs

Method	Path	Purpose	Scope
POST	`/api/v1/reservation-request`	Create a reservation or custom request	`create_reservation_request`
GET	`/api/v1/customer/reservation-requests`	List my requests	`requester_view_reservation_request`
GET	`/api/v1/customer/reservation-request/{id}`	Request detail	same
POST	`/api/v1/reservation-request/{id}/cancel`	Cancel	`requester_manage_reservation_request`
POST	`/api/v1/reservation-request/{id}/offering/{oid}/extend-acceptance-expiry`	Extend acceptance window	`accept_contract_extension`
POST	`/api/v1/reservation-request/{id}/offering/{oid}/extend-contract-expiry`	Extend contract window	`accept_contract_extension`
GET	`/api/v1/gpu-offerings`	Browse offerings	`view_capacity`
GET	`/api/v1/gpu-offerings/availability`	Availability	`view_capacity`
GET	`/api/v1/gpu-offerings/capacity-overview`	Capacity overview	`view_capacity`
GET	`/api/v2/gpu-offerings/availability-by-filters`	Filtered availability	`view_capacity`
GET	`/api/v1/organizations/suggest`	Organization name suggestions	authenticated

Providers APIs (NCP and admin)

Method	Path	Purpose	Scope
GET	`/api/v1/provider/reservation-requests`	List requests for my NCP (all for admins)	`approver_view_reservation_request`
GET	`/api/v1/provider/reservation-request/{id}`	Detail	same
POST	`/api/v1/provider/offering/{id}/accept`	Accept offering	`approver_manage_reservation_request`
POST	`/api/v1/provider/offering/{id}/reject`	Reject (remarks required)	same
POST	`/api/v1/provider/offering/{id}/contract`	Confirm contract	same
POST	`/api/v1/provider/offering/{id}/resend-magic-link`	Resend magic link	provider/admin
GET	`/api/v1/provider/custom-demand-requests`	List custom requests	`approver_view_custom_reservation_request`
(status update)	custom demand transition endpoints	In Review / Matched / Unserviceable	`approver_manage_custom_reservation_request`
GET	`/api/v1/provider/gpu-offerings`	My inventory	`view_provider_inventory`
POST	`/api/v1/provider/gpu-offerings/import`	Upload capacity CSV (202)	`manage_inventory`
GET	`/api/v1/provider/gpu-offerings/import/history`	Upload history	`manage_inventory` / admin
POST	`/api/v1/provider/gpu-offerings/import/{id}/approve`	Accept future capacity	`review_capacity`
POST	`/api/v1/provider/gpu-offerings/import/{id}/reject`	Reject future capacity	`review_capacity`
POST	`/api/v1/provider/onboard-providers`	Bulk onboard NCPs (CSV)	`manage_ncp`
GET	`/api/v1/provider/global-capacity-map`	Global capacity	`global_capacity_view`

Access Request APIs (only when NPN integration is enabled, otherwise 404)

Method	Path	Scope
GET	`/api/v1/provider/access-requests`	`manage_ncp`
GET	`/api/v1/provider/access-requests/{id}`	`manage_ncp`
POST	`/api/v1/provider/access-requests/{id}/approve`	`manage_ncp`
POST	`/api/v1/provider/access-requests/{id}/reject`	`manage_ncp`

Inventory APIs (reference data)

Countries, geographical regions, GPU models, hosting providers, interconnect types, regions — read with `view_capacity`; create geographical region or GPU model with `manage_inventory`.

Users APIs

Method	Path	Purpose
POST	`/api/v1/user`	Provision the signed-in user from JWT claims (organization, partner domain checks; may create an access request)
GET	`/api/v1/user`	Current user
POST	`/api/v1/users/me/terms-acceptance`	Record Terms and Conditions acceptance
POST	`/api/v1/auth/logout`	Logout

Operations APIs (role `operations`)

Activate user, list users, resend activation magic link, delete user.

Feedback APIs

Submit feedback, check eligibility (`/api/v1/feedback/eligibility`), record prompt state (`/api/v1/feedback/prompt`). Feedback kinds: `POST_TASK` , `GENERAL` , `CANCELLATION` , `DROP_OFF` . Journeys: `capacity_upload` , `reservation_request` , `app_general` . Categories: `suggest_feature` , `report_problem` , `general` . Prompt states: `SHOWN` , `SUBMITTED` , `SKIPPED` , `DISMISSED` .

System APIs (Temporal worker service account)

Group	Scope
`/api/v1/notification/*` (all emails, reminders, magic links)	`notification:send`

Group	Scope
`/api/v1/multi-provider/expire-acceptance`, `/expire-contract`	`notification:send`
`/api/v1/reservation-request/expire`	`reservation-request:write`
`/api/v1/offering/validate-nonce`	Keycloak extension
`/api/v1/system/csv-import/*`, `/api/v1/system/provider-onboarding/process`	`capacity:sync`

Internal APIs (hidden from schema)

- `POST /api/v1/internal/salesforce/offtake-sync`, `/notify-failure`, `/partner-domain-sync` — caller `client_id` must equal `SFDC_SYNC_CALLER_CLIENT_ID`.
- `POST /api/v1/internal/audit/keycloak-event` — Keycloak audit listener ingest.

Cross-cutting behaviour

- JWT validation against Keycloak; permissions from `scope`, `realm_access.roles` and `resource_access.*.roles` claims.
- Middleware blocks users who are PENDING or have not accepted terms (with an allowlist for the endpoints needed to finish setup).
- Request body limit 15 MB; CSV upload limit 10 MB.
- Redis-backed rate limiting per bucket.

Compute Match — Temporal Workflows and Schedules

Workflows are defined in `shared/workflows/` and run by the worker in `workflow_worker/` (`main.py` registers workflows, activities and schedules). Activities call the backend System APIs; the backend owns the domain logic.

Summary

Workflow	Trigger	Business purpose
`ReservationRequestWorkflow`	Backend, when a request is created (workflow id = request id)	Orchestrates a reservation request end to end
`OfferingLifecycleWorkflow`	Child of the above, one per routed NCP offering	Magic link, acceptance window, contract window, reminders
`CsvImportWorkflow`	Backend, on capacity CSV upload (`csv-import-{job_id}`)	Validate, import, wait for admin review of future capacity
`ProviderOnboardingWorkflow`	Backend, on NCP onboarding CSV upload	Create NCP organizations and users
`InProcessAlertWorkflow`	Schedule `in-process-requests-alert-daily` , cron `0 9 * * *` UTC	Daily admin digest of In Process offerings near deadline
`AuditRetentionWorkflow`	Schedule `audit-retention-daily` , cron `30 3 * * *` UTC	Delete audit log rows older than about 395 days
`SfdcCreateBatchWorkflow` (mode `create`)	Schedule, default `0 5 * * *` UTC	Create Salesforce Offtake records from the outbox
`SfdcCreateBatchWorkflow` (mode `status`)	Schedule, every 2 minutes	Push status updates to existing Offtakes
`SfdcPartnerDomainSyncWorkflow`	Schedule `20 2,14 * * *` (twice daily)	Refresh partner email domains from Salesforce

ReservationRequestWorkflow

1. Send “request received” email to the customer.
2. Wait for signal `routed_to_providers` or `reservation_request_closed` . Open/custom requests wait indefinitely.
3. On routing, start one child `OfferingLifecycleWorkflow` per offering.
4. Wait until any child is Contracted, all children finish, or the parent is closed.
5. If one contracts: cancel siblings (superseded) and approve the request.
6. If all expire: send one consolidated customer expiry email and expire the request.
7. Closure reasons send emails: Cancelled (customer + NCPs), Rejected, Matched, Unserviceable.

Signals: `routed_to_providers` , `reservation_request_closed` . Child workflow retry up to 100 attempts; expire activity 20 attempts over about 75 minutes.

OfferingLifecycleWorkflow

1. Send the magic-link email (critical retry policy).
2. Acceptance phase (7 days by default): reminder 3 days before; on timeout expire the offering and email the NCP.
3. On `provider_accepted`: send the introduction email; start the contract phase (45 days).
4. Reminder 3 days before contract expiry; on timeout expire and email.
5. On `offering_contracted`: send contracted email. On reject or cancel: handle and email (superseded if a sibling won).

Signals: `provider_accepted`, `acceptance_expiry_extended`, `contract_expiry_extended`, `offering_contracted`, `offering_rejected`, `offering_cancelled`.

CsvImportWorkflow

VALIDATING → IMPORTING → (if future rows) PENDING_REVIEW: email admins and wait for `approve_import` / `reject_import` → activate or reject offerings → email the NCP. Activities: `validate_csv`, `execute_atomic_import`, `activate_offerings`, `reject_offerings`, `update_import_job_status`, `notify_admin_for_review`, `notify_provider_result`. Workflow max attempts 1; import activities 3 attempts.

ProviderOnboardingWorkflow

`validate_provider_csv` → `execute_provider_onboarding` (30-minute timeout) → `notify_onboarding_complete` (sent even on failure). 3 attempts; validation errors are not retried.

Salesforce workflows

- `SfdcCreateBatchWorkflow` drains the `salesforce_create_sync` outbox. **Create** mode retries 13 times every 5 minutes; **status** mode runs once per tick. Gated by `SFDC_SYNC_ENABLED`; when false, schedules are deleted and the workflow is not registered.
- `SfdcPartnerDomainSyncWorkflow` is gated by `SFDC_PARTNER_DOMAIN_SYNC_ENABLED`. It refuses empty or suspiciously shrunk snapshots so partners are not mass-demoted.

Retry policies (`shared/workflows/workflow_common/retry.py`)

Policy	Budget	Used for
`NOTIFICATION_RETRY_POLICY`	5 attempts, ~2.5 min	Ordinary email
`CRITICAL_NOTIFICATION_RETRY_POLICY`	10 attempts, ~25 min	Magic-link email
`STATUS_UPDATE_RETRY_POLICY`	5 attempts, ~1 min	Intermediate import status
`TERMINAL_STATUS_UPDATE_RETRY_POLICY`	8 attempts, ~20 min	Final import status

Abandoned steps increment `clustruffle_workflow_activity_abandoned` and can be queried with `get_abandoned_steps`.

Viewing workflows locally

Temporal UI: `http://localhost:7600`. Temporal gRPC: `localhost:7233`. Run the worker on your laptop with `make worker`.

Compute Match — Identity and Keycloak

Realm and clients

- Single realm: `customer` (organizations enabled). Older provider/operations realm files exist but the application uses one unified realm.
- Local realm export: `keycloak/realms/local-realm-customer.json`.
- Main clients: `customer-web` (frontend OIDC), `workflow-worker` (Temporal service account), `audit-listener`, `superset`, `swagger`, `temporal-ui`.
- Identity broker **Starfleet** (NVIDIA SSO) with a `starfleet-roles` mapper. The frontend sends `kc_idp_hint=starfleet`.

Roles

`customer`, `provider`, `provider_business_user`, `provider_basic_user`, `platform_business_admin`, `platform_business_admin_super_user`, `platform_business_admin_ops_user`, `account_manager_consumer`, `account_manager_provider`, `superset_users`, `superset_admins`, `superset_dashboard_viewers`, plus the composite `default-roles-customer`.

Groups used by the app: `offering-validator` (magic-link recipients), `mailing-list`.

Custom extensions (`keycloak-extensions/`)

Extension	What it does
`accept-reservation-token`	Custom action token for magic links: validates the JWT, blocks a different logged-in user, validates the nonce with the backend, creates the session and redirects. Exposes the token-minting REST endpoint with a rate limiter
`audit-event-listener`	Sends LOGIN and LOGOUT events (and errors) to the backend `/api/v1/internal/audit/keycloak-event`
`default-role-restriction`	Self-signup users keep `default-roles-customer`; admin-created users have defaults stripped
`multi-org-policy`	Only account-manager roles may belong to more than one organization

Build with `keycloak-extensions/build.sh`; images from `keycloak/Dockerfile` and `keycloak/Dockerfile-local`.

Themes

The NVIDIA theme lives in `keycloak/themes/nvidia/` (login, account, admin, email). The login theme extends `keycloak.v2` and adds organization-type registration fields (enterprise, government, sme_msme, startup).

Organizations

- Keycloak Organizations model NCP and customer companies.
- NCP organization names come from the Salesforce account name; one organization per Salesforce account.
- The `organization` JWT claim carries the user's organization; the backend stores matching organization records.

Local Keycloak

- Admin console: `http://localhost:8080/admin`, realm `customer`.
- Management/health port: host 9001.
- Admin credentials are generated into `local-setup/.onboarding/credentials.json`.

Compute Match — Analytics with Superset

Apache Superset (`superset/`) provides dashboards for Platform Admins. Locally, start it with `make onboard ONBOARD_ARGS="--with-analytics"` and open `http://localhost:8088` with the operations demo account.

Access

- Custom role **DashboardViewer**: can view dashboards, charts and datasets in the `analytics` schema only; no SQL Lab, no query editing, no database admin.
- Keycloak role `superset_dashboard_viewers` (part of the Platform Admin composite) maps to DashboardViewer (`superset/superset_config.py` , `superset/custom_roles.py`).
- In the app, Platform Admins open it via the external **Dashboard** menu item.

Dashboard 1 — Compute MatchMake System Metrics (marketplace)

Defined in `superset/dashboards/marketplace_metrics.py` .

- **Request analytics**: total requests, requests per week, average time to contract (overall and by NCP), conversion rate (overall and by GPU), acceptance and rejection by NCP, custom-request percentage, contracted GPUs by month and platform.
- **User analytics**: NCP and customer activity tiers with detail tables, new sign-ups in the last 4 weeks, logins per user over 90 days.
- **Capacity analytics**: available, reserved and future GPUs, NCPs with capacity, capacity by platform and region, future pipeline by month.

Sources: views such as `analytics.v_reservation_request_*` , `v_ncp_activity` , `v_customer_activity` , `v_user_actions` , `v_ncp_capacity_overview` , `v_future_pipeline_by_month` .

Dashboard 2 — Feedback

Defined in `superset/dashboards/feedback_metrics.py` . KPIs: average rating, number of responses, satisfied versus dissatisfied, rating distribution, top reasons, journey summary, recent feedback. Filters: time, persona, journey. Source: `analytics.v_feedback` and a funnel view.

Compute Match — Local Development Guide

Prerequisites

- Git, Bash, Make, Docker Desktop with Compose v2, and NVIDIA network/VPN access.
- Either **Devbox** 0.18.3 or newer (supplies all tools) or installed tools: Node 22 (frontend), Python 3.13 and `uv` (backend and worker).
- Docker-only mode (`make onboard`, `make start`) needs no Node or Python on your laptop.

First run

```
make onboard      # build containers, prepare databases and identity, create demo accounts and inventory
make verify       # should finish with "status": "ready"
```

The first build can take 10–20 minutes. Without Make, run `./local-setup/onboard.sh`. If it fails, read `local-setup/.onboarding/setup.log`, fix the problem and rerun.

Open `http://localhost:3000`, click **Login**, enter a demo account email, then its generated password.

Demo accounts

Passwords are generated into `local-setup/.onboarding/accounts.md` (human guide) and `credentials.json` (machine-readable, includes the Keycloak admin login). These files are Git-ignored.

Experience	Account
Customer	`developer@customer-a.example.org`
Provider (NCP)	`seed@singtel.example.org`
Platform business admin	`business-admin@platform.example.org`
Operations	`ops@platform.example.org`

Normal onboarding creates 17 accounts; Singtel and Firmus provider accounts test isolation between NCPs, and colleague accounts test shared inventory inside one NCP. `make onboard ONBOARD_ARGS="--with-scenarios"` adds 22-account regression personas, including `pending@scenarios.example.org` (pending activation), `terms@scenarios.example.org` (terms not accepted), `restricted@scenarios.example.org` (active, no business role), `infra-manager@customer-a.example.org` and `tech-admin@customer-b.example.org`.

Test sign-ups with addresses such as `you@example.com`; `example.com` and `nvidia.com` auto-activate, personal domains are blocked, and local email delivery is disabled. A new sign-up may need a role assigned in Keycloak (**customer** realm → user → **Role mapping**), then sign out and in again.

`make onboard ONBOARD_ARGS="--no-seed"` creates accounts without demo inventory. Sample CSVs for upload testing are in `local-setup/fixtures/`.

Development modes

Command	Runs on your laptop	Runs in Docker
`make start`	nothing	frontend, backend, worker
`make frontend`	frontend	backend, worker
`make backend`	backend	frontend, worker
`make worker`	worker	frontend, backend
`make dev` (fullstack)	all three	none

PostgreSQL, Keycloak, Temporal and Redis always stay in Docker. Install dependencies for a mode with `make deps MODE=backend` (or `frontend`, `worker`).

Make targets

Target	Purpose
`make help`	List commands
`make onboard`	Configure, start and seed (`ONBOARD_ARGS` for options)
`make doctor`	Check prerequisites and configuration only
`make deps`	Install locked dependencies for a `MODE`
`make build`	Build local images
`make start` / `make stop`	Start all in Docker / stop everything but keep data
`make start-infra`	Start only supporting containers
`make status`	Show mode, processes and URLs
`make logs SERVICE=backend`	Follow logs (frontend, backend or worker)
`make verify`	Check accounts, permissions, inventory, login routing and worker polling
`make test`, `make test-backend`, `make test-frontend`, …	Unit tests
`make quality`	Lint, type checks and tests
`make clean`	Remove containers, volumes and images
`make obs-start` / `obs-stop` / `obs-logs`	Optional SigNoz telemetry stack

Local URLs and ports

Service	Address
Application	http://localhost:3000
Backend API docs (Swagger)	http://localhost:8082/docs
Keycloak	http://localhost:8080 (admin at `/admin`, realm `customer`)
Temporal UI	http://localhost:7600
Temporal gRPC	localhost:7233
PostgreSQL	localhost:5432
Redis	localhost:6379
Superset (optional)	http://localhost:8088
SigNoz (optional)	http://localhost:3301 (OTLP gRPC 4317, HTTP 4318)

Configuration files

Location	Use
`local-setup/.env`	Generated service settings and secrets
`local-setup/.onboarding/credentials.json`	Account credentials
`local-setup/.onboarding/accounts.md`	Account guide
`local-setup/.onboarding/dev/`	Host-mode settings, process state and logs
`local-setup/.onboarding/setup.log`	Onboarding diagnostics

You do not need to copy `.env.sample` or create service dotenv files.

Everyday loop

`make stop` at the end of the day keeps your data; `make start` resumes. Use `make status`, then `make logs SERVICE=...` when something looks wrong.

Common problems

Problem	Fix
Image or Maven downloads fail	Connect to the NVIDIA VPN
Port already in use	Stop the other checkout or process using the port
Wrong Node version	Use Node 22 or set `LOCAL_DEV_NODE`
Login redirects to a Docker hostname	Restore localhost public URLs in the generated settings
Signed up but see no features	Assign a realm role in Keycloak and sign in again

Problem	Fix
SigNoz shows no data	Finish SigNoz first-user setup, run `make obs-start`, restart apps with tracing; Docker Engine 25+ for container metrics
Everything is broken	`make clean && make onboard && make verify`

Compute Match — Environments, Infrastructure, Delivery and CI

Environments

Environment	How applications are delivered
Dev	Continuously from protected `main`
QA	Manual promotion of a qualified Dev snapshot
Staging (STG)	Manual promotion of the immutable QA package (`STG_RELEASE_SNAPSHOT_ID`)
Production	Not enabled from the pipeline until authorized

Infrastructure for each environment is applied by a separate Terraform pipeline. Environment values live in `infrastructure/environments/<env>/`.

Google Cloud infrastructure (Terraform)

Stacks in `infrastructure/terraform/stacks/`, applied in order: `project-bootstrap` → `environment-bootstrap` → `computematch` → `platform` → `keycloak`.

- **computematch** stack: VPC, **GKE Autopilot** private cluster, **Cloud SQL PostgreSQL**, **Memorystore Redis**, Secret Manager metadata, Workload Identity, edge and Cloud Armor.
- **platform** stack: namespaces, Kubernetes service accounts, RBAC, NetworkPolicy baselines, Gateway resources.
- **keycloak** stack: declarative realm and identity-provider configuration.

Terraform owns cloud resources; Helm owns workloads. Runtime secrets live only in Google Secret Manager.

Helm

Charts in `infrastructure/helm/`: workloads `backend`, `frontend`, `workflow-worker`, `keycloak`, `superset`; platform `db-bootstrap`, `otel-collector`; library `common`. Deployment order: db-bootstrap → otel-collector → keycloak → backend → worker → frontend → superset. Workloads use SecretSync and Cloud SQL Auth Proxy sidecars; image digests are pinned in environment values.

Application release train

“Build once, promote digests.” Components: backend, frontend, worker, superset, keycloak.

1. Merge to `main` publishes images tagged by commit SHA and mirrors them to Artifact Registry.
2. Dev auto-deploys changed components, runs smoke and functional checks, and records a snapshot.

3. QA promotes the exact digests and creates the package `computematch-release/<snapshot-id>` (manifest plus checksum).
4. Staging consumes that package without rebuilding.
5. Production promotion is not yet enabled.

GitLab CI

Stages: test → security → build → publish → sync → plan → infrastructure → platform → deploy → verify.

- Merge requests: path-scoped tests, security scans (secret scan, IaC SAST), image builds without publishing, read-only Terraform plans.
- `main`: publish images, mirror, Dev release. Infrastructure applies are manual on protected `main` and independent of application releases.

Git workflow (CONTRIBUTING.md)

- Branch: `NCPMS-<number>/<kebab-case-outcome>`, for example `NCPMS-565/repository-foundation`. One Jira issue per branch; branch from updated `main`; no long-lived develop/staging branches.
- Commit: `<type>(<scope>): NCPMS-<number> <imperative description>`, for example `fix(backend): NCPMS-855 widen hosting provider name columns`. Types: feat, fix, docs, test, refactor, perf, build, ci, chore, revert. Add `!` for breaking changes.
- Merge request title: `NCPMS-<number>: <Capitalized outcome>`. Description sections: Objective, Changes, Validation, Risk and rollback, Tracking.
- Labels: one `type::`, one `risk::` (low, medium, high), one or more `component:`; optional `review:security`, `review:database`, `review:platform`, `breaking-change`.
- No direct pushes or force pushes to `main`; pipelines must pass and discussions be resolved; fast-forward merges; no whole-MR squash.
- Install hooks with `pre-commit install` and run `pre-commit run --all-files`.

Compute Match — Security, Observability, Salesforce Integration and Testing

Security model

- **Delivery identity:** GitLab OIDC with Google Workload Identity Federation; separate Google service accounts for plan, apply, deploy, verify and Keycloak.
- **Runtime identity:** Kubernetes service accounts mapped to Google service accounts; secrets from Secret Manager with least privilege; Cloud SQL Auth Proxy; separate database roles (for example the audit-retention role can only SELECT and DELETE).
- **Network:** NetworkPolicies, private Cloud SQL and Redis, Cloud Armor and Gateway at the edge.
- **Application:** Keycloak JWTs with audiences per client (`customer-web` , `workflow-worker` , `audit-listener` …). Internal Salesforce and audit endpoints only accept a pinned caller `client_id` . CSV cells are sanitized against formula injection. Personal email domains are blocked.
- **Repository:** protected `main` , CODEOWNERS, secret scanning (gitleaks, trufflehog, talisman), no secrets in manifests or docs.
- **Audit:** Keycloak login/logout events and domain actions are written to an audit log, kept about 13 months then purged by `AuditRetentionWorkflow` .

Observability

- **Logs:** every owned service writes JSON Lines to stdout using **Log Record v2** (`docs/architecture/logging/log-record-v2.md`): `log_schema_version: 2` , severity, `event_name` , resource, source, optional trace context, attributes and exception. Unsafe fields are redacted; no PII or secrets in attributes.
- **Event catalog** (`docs/architecture/logging/event-catalog.md`): for example `workflow.started` , `workflow.completed` , `workflow.failed` , `reservation.request.*` , `csv_import.*` , `provider_onboarding.*` , `integration.salesforce.failed` , `notification.delivery.failed` , `authorization.denied` .
- **Traces and metrics:** OpenTelemetry in frontend, backend, worker (Temporal TracingInterceptor) and Keycloak. In GCP the Helm `otel-collector` exports to Google Cloud; logs go to Cloud Logging. Locally, optional SigNoz (`make obs-start` , UI on port 3301).
- Runbooks: `docs/operations/*-observability.md` for backend, frontend, worker, Keycloak, Superset and gateway.

Salesforce integration (ADR-001)

- **Direction:** one-way, Compute Match → Salesforce **Offtake** object (`Offtake_Request__c`).
- **Pattern:** transactional outbox table `salesforce_create_sync` , drained by Temporal `SfdcCreateBatchWorkflow` which calls backend internal APIs (domain stays in the backend). No Kafka or Pub/Sub.
- **Granularity:** one Offtake per NCP offering (external id `Reservation_Request_Offering_Id__c`); open custom requests get one create-only Offtake keyed by the request id.
- **Jobs:** nightly **create** (05:00 UTC, full projection) and **status** every 2 minutes (status and, when Contracted, negotiated terms). Upserts use the sObject Collections external-id API, up to 200 records per call.

- **Reliability:** outbox states PENDING, SYNCED, FAILED; retries for Salesforce outages; mandatory failure summary email; audit events `SFDC_OFFTAKE_SYNCED` / `SFDC_OFFTAKE_SYNC_FAILED` written atomically with the outbox.
- **Toggles:** `SFDC_SYNC_ENABLED`; partner-domain read sync separately with `SFDC_PARTNER_DOMAIN_SYNC_ENABLED`.

Testing

Layer	Tooling	Command
Backend	pytest, TestContainers	`make test-backend`
Frontend	Jest 30, React Testing Library, jest-axe	`make test-frontend` (`npm run test`, `test:a11y`, `quality-gate`)
Workflows / worker	pytest	`make test`
End-to-end	Playwright + Cucumber (Gherkin) in `e2e_playwright/`	see its README

E2E features cover Authentication, CustomerWorkflow, ProviderWorkflow and AdminWorkflow, using page objects in `src/PageObjects/` and step definitions in `src/Specs/`. Choose the target with `TEST_ENV` (`local`, `computematch-dev`, `computematch-qa`, `computematch-stg`). Secrets go in a Git-ignored `.env.local`. HTML reports are written to `reports/`. `make verify` does not run the E2E suite.

Compute Match — Glossary, FAQ and Troubleshooting for Users

Glossary

Term	Meaning
Compute Match / Compute MatchMake System	NVIDIA's marketplace matching GPU capacity seekers with NVIDIA Cloud Providers
NCPMS	Jira project key and internal name
Clustruffle	Internal code name seen in metrics and Keycloak endpoints
NCP	NVIDIA Cloud Provider (GPU Compute Provider) that publishes capacity
Customer / Offtaker	Organization reserving GPU capacity
Platform Admin	NVIDIA admin who reviews capacity, onboards NCPs, brokers custom requests
NPN	NVIDIA Partner Network; NPN portal members can sign in directly
Starfleet	NVIDIA single sign-on service
Platform	GPU model, for example H100 or B200
GPU Offering / GPU Capacity	A cluster published by an NCP
Hosting provider	The NCP's data-center brand that hosts the offering
Region	A specific data-center location
Geographical Region	A broad area like North America or Europe
Current capacity (AVAILABLE)	Capacity starting within 90 days of upload; goes live immediately
Future capacity (FUTURE)	Capacity starting more than 90 days out; needs admin review
Live Token Date	CSV column for when capacity becomes available
Earliest Available Date	First date a cluster can be reserved from
Tenure	Reservation length band in years: 0-1, 1-2, 2-3, 3-4, 4-5, 5-6, 6+
Reservation Request	Customer's request to reserve capacity
Multi-NCP request	One request sent to several NCPs as alternatives; first to contract wins
Custom Request	Request with no specific cluster, brokered by admins

Term	Meaning
Offering (in a request)	One NCP's slice of a reservation request
Proposal	A version of the reservation terms; counteroffers add proposals
Counteroffer	NCP proposes different dates, GPU count, platform or region
Magic link	Secure one-click email link letting an NCP open and accept a request
Acceptance window	7 days for an NCP to accept
Contract negotiation window	45 days from acceptance to confirm the contract
Superseded	Another NCP contracted the same request, so yours was cancelled
Allocation	Capacity consumed by a contracted request
Offtake	Salesforce record for a deal synced from Compute Match
Access request	A partner's pending request for NCP access
offering-validator	Keycloak group whose members receive magic links
Confidential	Label shown when a customer's name is hidden from an NCP

Frequently asked questions

How do I sign up? Click Sign up or Login, enter your work email, and sign in through Starfleet. Personal emails are not allowed.

Why do I see "Access denied"? Your identity was verified but your account is not authorized under current policies. Contact computematchmake@nvidia.com or sign in with another work account.

Why do I see "Your access is being reviewed"? Your company is an NVIDIA partner but its NCP organization is not on the platform yet. A Platform Admin must approve your access request.

How many NCPs can I send one request to? Up to 3 in the UI by default (backend allows up to 10), if you have the multi-NCP permission.

Will I get several contracts if I send to several NCPs? No. The options are alternatives; the first NCP to confirm the contract wins and the others are cancelled.

How long does an NCP have to respond? 7 days to accept, then 45 days to contract. Reminders arrive 3 days before each deadline. Admins can extend both.

My magic link says it expired. Links last 1 hour. Ask for a resend (possible for 7 days after routing) or log in normally and open Manage reservation.

Can I accept an NCP counteroffer as a customer? The customer detail page is view and cancel only; the NCP confirms the contract after you agree with them directly.

When can I cancel? While the request is Awaiting NCP Acceptance, awaiting approval, In Process or Expired; custom requests while Open or In Review.

What if no cluster matches my needs? Submit a custom request. Admins will review and mark it Matched or Unserviceable.

My CSV upload shows Pending Review. It contains future capacity (starting more than 90 days out); a Platform Admin must accept it.

My CSV upload shows Validation Failed. Open “See all details” for row errors. Check the five required columns, UTF-8 encoding, 10 MB and 10,000-row limits.

Why can’t I see Upload Capacities or Manage reservation? You lack the scope (`manage_inventory` or `approver_manage_reservation_request`). Basic NCP users are read-only. Ask your Platform Admin.

I have customer and NCP access. How do I switch? Use **Switch profile to** at the top of the account menu.

Where is the user manual? Click the book icon in the header; it opens the manual filtered to your role. The `?` icon shows help for the current page.

What does What’s New show? Release highlights for your persona. A banner appears on your landing page for about 7 days; dismissing it keeps it under the bell icon. Use **Mark All as Read** to clear the unread dot.

How do I give feedback? Use **Share Feedback** in the header, or answer the star rating after creating a request or uploading capacity.

What’s New — October 2026 release (v1.3.0)

- Customers: faster, more compact request submission; real-time tracking of custom requests from Open to In Review to Matched.
- NCPs: clearer upload status and errors; sign in directly through the NPN portal.
- Admins: complete custom request workflow (Open → In Review → Matched or Unserviceable) in the admin view.